

ICE: Reducing Search Space for Error-Inducing Input Detection

Youshuai Tan , Zhanwei Zhang , Zishuo Ding , Jinfu Chen , and Weiyi Shang , *Senior Member, IEEE*

Abstract—Floating-point programs play a crucial role in scientific and engineering applications as well as daily life. Due to the inherent inaccuracies of floating-point arithmetic, these programs are susceptible to errors. While only a small subset of possible inputs may lead to significant inaccuracies in the program's final results, such errors can have serious consequences. Prior approaches have been developed to identify error-inducing inputs using search algorithms. However, the task is challenging due to the sparsity of the search space. Furthermore, these tools frequently depend on high-precision programs as oracles for computing floating-point errors. This reliance introduces additional challenges, including prolonged execution times and instability. To detect these inputs more effectively and efficiently, we propose a simple yet effective method called ICE, designed to generate reduced and erroneous domains for these approaches. ICE first calculates the errors of the target program by taking all half-precision floating-point numbers as inputs. Instead of calculating errors based on high-precision programs, ICE injects small perturbations into each atomic operation of the program and measures the difference between the perturbed and original results to estimate floating-point errors. Subsequently, ICE selects the numbers with errors in the top $n\%$ as the endpoints of the important ranges. These individual ranges, which form domains, can then be used to improve existing tools by reducing the search space. To evaluate ICE, we apply ICE with three state-of-the-art error-inducing input detection approaches, i.e., ATOMU, HSED, and FPCC. Experimental results demonstrate that ICE could improve the performance of existing approaches by effectively reducing the search space, detecting more error-inducing inputs, and significantly shortening the search time.

Index Terms—Floating-point error, dynamic testing, perturbation, condition number.

I. INTRODUCTION

FLOATING-POINT programs are significant in numerous fields of science and engineering, as well as in our daily life [1], [2], [3], [4], [5], [6]. Due to their inherent approximation

of real numbers and arithmetic, floating-point programs are inevitably prone to errors. Although only a small fraction of inputs may lead to substantial errors [7], these errors can result in severe consequences, such as rocket launch failure, stock market disorder, and the loss of life [8]. Such errors may also be significant in deep learning models, potentially causing the model to crash and leading to serious consequences [9], [10], [11], [12], such as the production of distorted outputs. However, it is extremely difficult for developers of such programs to identify which inputs will cause these errors without execution.

Automated approaches have been proposed to identify inputs to floating-point programs that would result in substantial errors [8], [13], [14], [15], [16], [17], [18], [19]. Such approaches are typically designed to utilize search-based algorithms. The typical design of existing approaches leads to two general limitations: 1) The floating-point number space is too sparse, which makes these approaches easily overlook significant numerical errors and allocate resources to unproductive space. 2) Most existing approaches rely on high-precision programs that employ higher numerical precision than the target program to compute reference outputs, thereby enabling error calculation and facilitating the search process. Existing approaches tend to use libraries, such as MPFR [20] and mpmath [21], to transform the original programs to high-precision programs, leading to an excessively long runtime and instabilities [22].

To address the above two limitations, we propose a novel method, named ICE, which aims to provide reduced domains to enhance existing automated tools. **Addressing limitation 1:** The first limitation lies in the fact that current tools overlook the removal of non-essential search spaces prior to initiating error search procedures. One intuitive way to overcome this limitation is to offer a more constrained search space. Specifically, ICE first takes all the half-precision floating-point numbers as grid points. Subsequently, we select grid points with floating-point errors in the top $n\%$ as endpoints for constructing the new domains. **Addressing limitation 2:** The second limitation consists of two drawbacks of using high-precision programs to calculate errors. Thus we propose a perturbation-based approach to simulate errors. Since floating-point errors stem from large atomic condition numbers among the program [8], [23], the given program's result may be significantly changed if we inject small perturbations into all the atomic operations (such as +). ICE simulates the errors by calculating the results' differences between the original programs and their corresponding perturbed versions. Only introducing perturbations during the

Received 27 July 2025; revised 13 April 2026; accepted 9 June 2026. Date of publication 15 June 2026; date of current version 21 September 2026. Recommended for acceptance by C. Menghi. (Corresponding author: Zishuo Ding.)

Youshuai Tan, Zhanwei Zhang, and Zishuo Ding are with the Data Science and Analytics Thrust, Information Hub, The Hong Kong University of Science and Technology (Guangzhou), Guangzhou 511400, China.

Jinfu Chen is with the School of Computer Science, Wuhan University, Wuhan 430072, China.

Weiyi Shang is with the Department of Electrical and Computer Engineering, University of Waterloo, Waterloo, ON N2L 3G1, Canada.

Digital Object Identifier 10.1109/TSE.2026.3703362

computation process, the simulation of errors is rather fast and efficient; whereas high-precision program-based approach can lead to an increase in time consumption by two orders of magnitude [22]. Moreover, based on condition number theory, the simulated error is deemed to be stable.

We evaluate ICE by applying it with three state-of-the-art tools, i.e., ATOMU [8], HSED [19], and FPCC [23]. Experimental results indicate that ICE can improve performance by effectively narrowing the search space, identifying more error-inducing inputs, and considerably reducing processing time. Specifically, ICE enables ATOMU to identify 29 additional error-inducing inputs, reflecting an increase of 93.54%. In addition, ICE helps uncover 20 more functions with error-inducing errors, representing a growth of 125%. However, since the runtime of ATOMU is independent of the search space size, ICE does not contribute to reducing its execution time. The application of ICE significantly reduces the runtime of HSED's first layer by 82.67%, maintaining the same number of detected error-inducing inputs. Similarly, FPCC achieves comparable results with a significant reduction in processing time by 97.82%. We also demonstrate that our approach to estimating errors is superior to the traditional method based on high-precision programs. The replication package, including the datasets and scripts, is available on the link¹.

In summary, our work makes the following three contributions:

- Our work introduces a simple yet effective method called ICE, designed to efficiently generate shortened domains that encompass error-inducing inputs. This capability enhances the performance of existing tools aimed at identifying error-inducing inputs.
- We conduct experiments by integrating ICE with three state-of-the-art error-inducing input detection tools. These experiments confirm that ICE not only accelerates the error identification process but also maintains or improves their ability to detect error-inducing inputs.
- We have open-sourced our tool, ICE, with the goal of benefiting the research community.

Our work sheds light on the unexplored potential of relying on error simulation to ensure the quality of floating-point programs, as an effective and efficient alternative to high-precision program-based errors.

Paper organization. The remainder of the paper is organized as follows. Section II presents the background of our work. We introduce the motivation in Section III. Section IV elaborates on ICE. The evaluation of our method is presented in Section V. In Section VI, we conduct ablation studies and incorporate multi-input functions into the evaluation. We discuss limitations and introduce related work in Section VII and Section VIII, respectively. Finally, Section IX concludes our work.

II. BACKGROUND

In this section, we present relevant background information on floating-point representation, error measurement, and condition number.

A. Floating Point Representation

By the mid-1950s, floating point computation had become widely adopted. The finite IEEE floating point numbers can be divided into three parts and the standard form is [24]:

$$\pm(b_0.b_1b_2\dots b_{p-1})_2 \times 2^E, \text{ where} \quad (1)$$

- $\pm$ is the sign part, determining positive or negative;
- p is the precision of the floating point system, which denotes the number of bits in the significand part. With respect to the normalized numbers, b_0 is 1;
- E is the exponent part based on m bits and $E = e - bias$, where e is the biased exponent value and $bias = 2^{m-1} - 1$.

Machine epsilon is an important concept, which represents the gap between 1 and its next floating point number as:

$$\epsilon = (0.00\dots 01)_2 = 2^{-(p-1)}. \quad (2)$$

More generally, for a floating point number x , we have

$$ULP(x) = (0.00\dots 01)_2 \times 2^E = 2^{-(p-1)} \times 2^E = \epsilon \times 2^E, \quad (3)$$

where ULP is an abbreviation of Unit in the Last Place, indicating the gap between x and the next floating number.

B. Error Measurement

Due to the inherent approximation of floating point numbers and arithmetic [25], the results of floating point programs often exhibit errors compared to real number computations. Let x represents a real number and $\hat{x}$ denotes its floating-point approximation. For a floating program $\mathbf{P}: \hat{y} = \hat{f}(\hat{\mathbf{x}})$, $\hat{y}$ refers to the numerical outcome and $f(\mathbf{x})$ represents for the real result. To quantify the error between the ideal mathematical result $f(\mathbf{x})$ and the outcome of the floating-point program $\hat{f}(\hat{\mathbf{x}})$, two commonly used metrics are:

$$\begin{aligned} Err_{abs}(f(\mathbf{x}), \hat{f}(\hat{\mathbf{x}})) &= |f(\mathbf{x}) - \hat{f}(\hat{\mathbf{x}})| \\ Err_{rel}(f(\mathbf{x}), \hat{f}(\hat{\mathbf{x}})) &= \left| \frac{f(\mathbf{x}) - \hat{f}(\hat{\mathbf{x}})}{f(\mathbf{x})} \right| \end{aligned} \quad (4)$$

where Err_{abs} and Err_{rel} denote the absolute error and relative error, respectively. However, when $f(x)$ approaches 0, the computation can become problematic due to division by zero. To address this problem, Err_{ulp} is often utilized:

$$Err_{ulp}(f(\mathbf{x}), \hat{f}(\hat{\mathbf{x}})) = \left| \frac{f(\mathbf{x}) - \hat{f}(\hat{\mathbf{x}})}{ULP(f(\mathbf{x}))} \right|. \quad (5)$$

Therefore, in this work, we use Err_{ulp} as our metric.

¹<https://anonymous.4open.science/r/ICE-4A35/>

C. Condition Number

The condition number quantifies the extent to which the relative rounding error in x is magnified when evaluating $f(x)$ at x [24]. Condition number is used in prior research as an indicator of searching inputs that lead to large errors [8], [23]. Its formula can be derived using the Taylor Expansion Theorem [8]:

$$\begin{aligned}
 Err_{rel}(f(x), f(x + \Delta x)) &= \left| \frac{f(x + \Delta x) - f(x)}{f(x)} \right| \\
 &= \left| \frac{f(x + \Delta x) - f(x)}{\Delta x} \cdot \frac{\Delta x}{f(x)} \right| \\
 &= \left| \left(f'(x) + \frac{f''(x + \theta \Delta x)}{2!} \Delta x \right) \cdot \frac{\Delta x}{f(x)} \right|, \theta \in (0, 1) \\
 &= \left| \frac{\Delta x}{x} \right| \cdot \left| \frac{x f'(x)}{f(x)} \right| + O((\Delta x)^2) \\
 &= Err_{rel}(x, x + \Delta x) \cdot \left| \frac{x f'(x)}{f(x)} \right| + O((\Delta x)^2) \quad (6)
 \end{aligned}$$

where Δx refers to small perturbation and θ is the remainder. As a result, the equation provides the formula for the condition number [26], which is given by $\left| \frac{x f'(x)}{f(x)} \right|$.

As the term $O((\Delta x)^2)$ could be ignored, we can conclude that a large condition number indicates that a small error in the input may lead to a substantial error in the output. We consider addition as an example: $x + y$. According to the formula for condition number, this atomic condition number with respect to x is given by $\left| \frac{x}{x+y} \right|$. Because of the inherent error in $\hat{x}$, the result would contain a large error when x approaches $-y$, where the phenomenon is called catastrophic cancelation.

D. Definition of Key Terms

To ensure clarity and precision, we define the key terms used in this study. Several tools, such as FPCC and ATOMU, employ search-based algorithms to identify inputs that induce large numerical errors in floating-point programs. Both FPCC and ATOMU detect inputs if the corresponding relative errors exceed 0.001. Accordingly, in this manuscript, we define inputs whose relative errors are greater than 0.001 as **error-inducing inputs**. Moreover, **erroneous domains** refer to areas in the input space where the likelihood of encountering error-inducing inputs is significantly higher than the average.

III. MOTIVATION

Although only a limited subset of inputs can trigger substantial floating-point errors [7], current state-of-the-art tools, including ATOMU, HSED, and FPCC, demonstrate limitations by initiating their search processes without reducing the search space. As a result, these tools not only incur substantial computational overhead but also risk overlooking numerous critical errors, thereby limiting their effectiveness.

Limitation 1: large search space with small erroneous domain. The first limitation of prior research is that most approaches act on the entire input value range as a large search

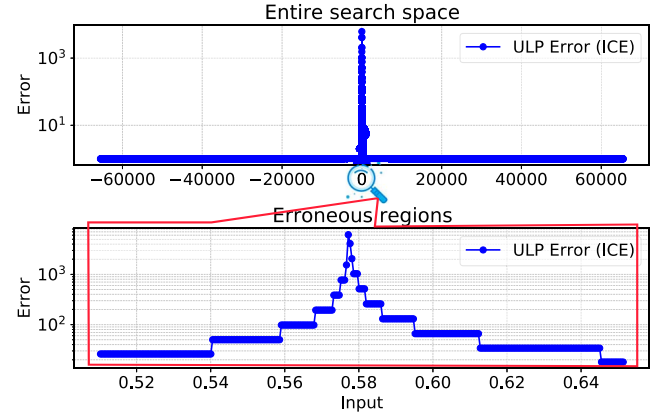


Fig. 1. Floating-point errors distribution of a real GSL function. Across the full search space, substantial errors appear exclusively in a limited zone near zero.

space. A compelling illustration of this limitation can be observed in Fig. 1, which demonstrates the error distribution of a real GSL function². Among most of the values in its input value range, only a small region near zero exhibits substantial errors throughout the entire search space; while all other input values would not lead to any substantial error. This example demonstrates that an unguided search over the entire input value range would cause prior approaches to spend excessive time finding the error-inducing inputs.

A straightforward approach to address this limitation that can be envisioned is to provide a smaller search space to the existing approaches. For example, one may take a very small sample of input values from the input value range and compute their corresponding errors. Such samples can be used to further estimate a reduced search space. In our proposed approach (detail presented in Section IV), we construct a grid over the input domain as sample input values and compute the errors for all grid points. Subsequently, erroneous domains can be constructed by selecting grid points with relatively large errors. **Limitation 2: depending on high-precision programs to calculate floating-point errors.** The second limitation of prior research lies in the fact that floating-point errors derived from high-precision programs may not be suitable for error-detection tasks.

Most prior works utilize high-precision programs to obtain oracles and calculate floating-point errors. However, we find that high-precision program-based approaches contain two drawbacks. First, high-precision programs are time-consuming. Specifically, quadruple precision (128 bits) is roughly 100 times slower than double precision (64 bits) [22]. Second, the errors computed by high-precision programs tend to be unstable. For instance, Fig. 2 illustrates the error distribution near the point 2.197141326031016 for the function `gsl_sf_bessel_Y1_e`, where the largest error occurs. The red points, representing results from high-precision programs, show varying error magnitudes, with some exhibiting only minor errors. Such instability

²https://www.gnu.org/software/gsl/doc/html/specfunc.html#c.gsl_sf_legendre_P2

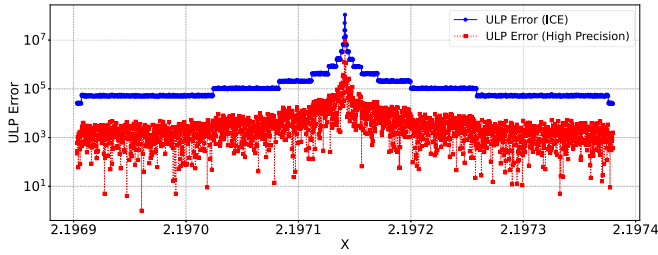


Fig. 2. Floating-point errors of points in the vicinity of one error-inducing input. A larger error value indicates a closer proximity to the error-inducing input.

is detrimental to the error-inducing inputs detection tools, such as HSED, because inputs within large errors may be missed. The instability can also impair the generation of erroneous domains, as it may lead to the exclusion of grid points which are located near the error-inducing inputs but exhibit minimal errors themselves.

To address the above two limitations, we propose our method, named ICE (presented in the next section).

IV. ICE: REDUCING SEARCH SPACE FOR ERROR-INDUCING INPUT DETECTION

The primary goal of our proposed method, ICE, is to generate erroneous domains composed of discontinuous intervals for floating-point programs. Existing tools such as ATOMU, HSED, and FPCC differ from ICE; they are designed to search for error-inducing inputs. ICE can significantly reduce the search space for these tools. Specifically, instead of traversing a single, continuous, and expansive input interval, these tools can be guided to focus exclusively on the discrete sub-intervals identified by ICE.

In particular, we first construct one grid and take all grid points as input arguments to compute floating-point errors within the given floating-point programs. We calculate the floating-point errors through a perturbation injection mechanism. Specifically, we inject perturbations into the floating-point programs, generating corresponding perturbed program variants. The discrepancies between the outputs of the original and perturbed programs are then quantified as simulated floating-point errors. Finally, we select the grid points within large floating-point errors to construct erroneous domains.

In this section, we first present the theoretical foundation underlying our simulation of floating-point errors, followed by a concrete demonstration of the process through a simple example. Furthermore, we conduct validation experiments to verify the reliability of our error simulation approach. Subsequently, we elaborate on our proposed method, which comprises three main steps. An overview of our method is shown in Fig. 3.

A. Simulating Floating-Point Errors

Before presenting the details of our approach ICE, in this subsection, we present the theoretical foundation of our approach to simulate floating-point errors, due to its importance.

To facilitate a clear understanding of the proposed approach, we show its details through an illustrative example. Finally, we experimentally validate its reliability.

Theoretical foundation of error simulation. The way we simulate floating-point errors is grounded in the theory of condition numbers. Floating-point errors stem from large condition numbers [8], [23], which amplify the inherent errors during computational processes. Specifically, $Err_{rel}(f(x), f(x + \Delta x))$ is equal to $Err_{rel}(x, x + \Delta x)$ multiplied by the condition number of $f(x)$, where $f(x)$ can be a simple operation in floating-point programs, such as addition (cf. Section II-C). Being a variable of a floating-point program, the inherent error of x is inevitable; while what really matters is how large the error is. Therefore, if the condition number of one operation is large, the operation's result will contain substantial error. It is worth noting that the condition number is computed by concrete operands of the operation, meaning that different input arguments of floating-point programs lead to varying condition numbers for all operations. To obtain the condition number of a specific operation, the entire floating-point program must be executed with a given input argument. As a consequence, we are not able to utilize static methods to calculate condition numbers of operations in the floating-point programs. Furthermore, although a large condition number of one operation results in substantial error of the operation, the error may not be propagated to the result of the floating-point program due to precision loss caused by the addition of large and small numbers. Thus, we simulate floating-point errors by injecting perturbations into all operations and compute the differences of two final results. Our design philosophy is that the difference is substantial if any operation contains a large condition number and the error can propagate to the final result.

Several existing studies [27], [28] introduce random perturbations into floating-point numbers within programs. If repeated executions yield significantly divergent results or exhibit high variance, the program is considered unstable. These works demonstrate a correlation between the deviation in results caused by perturbation and the inherent floating-point error of the program. In contrast to these approaches, our method differs in two key aspects: (1) the perturbations are applied deterministically to all operations, and (2) the method requires only a single execution of the perturbed program.

A simple example to show the process of simulated error calculation. Table I and Fig. 4 illustrates the key steps of our method to simulate floating-point errors, demonstrated through a representative example. The example consists of three operations, with an input argument of 0.304692654015397. We first subtract one ULP from the input to trigger the potential large condition number of the first operation before executing the perturbed program. For the first operation, the ULP difference is calculated as $\left| \frac{a-b}{ULP(a)} \right|$, where $a = 0.29999999999999954$ and $b = 0.29999999999999943$, yields a small result of 2.0. We utilize the high-precision program to calculate the oracle Err_{ulp} , resulting in 1.1340e-01. Therefore, our method calculates the error of the first operation correctly from the simulation perspective. As for the second operation, since we inject

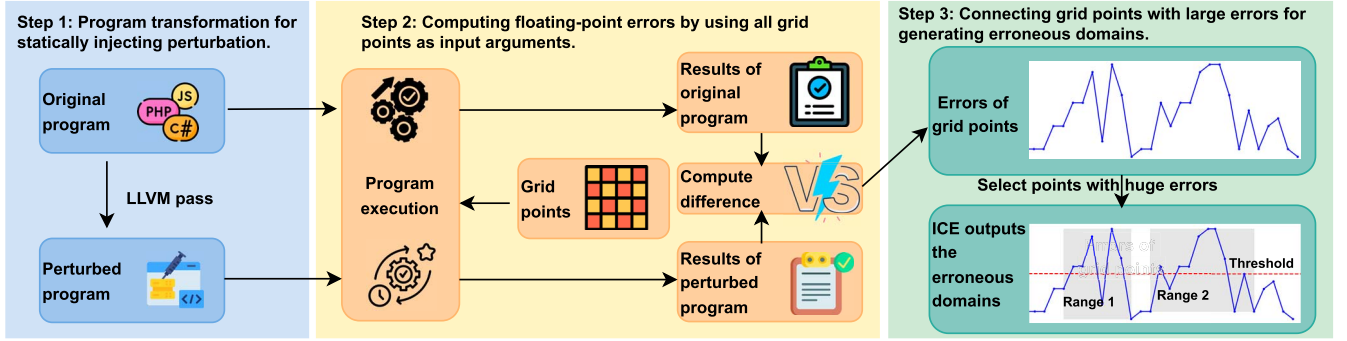


Fig. 3. An overview of our approach ICE.

TABLE I
AN EXAMPLE ILLUSTRATING THE ERROR SIMULATION OF OUR METHOD AND THE INPUT ARGUMENT IS 0.304692654015397

Operation	Original Program	Perturbed Program	Error from ICE	Error from high-precision program	Condition Number
$\sin(x)$	2.9999999999999954e-01	2.9999999999999943e-01	2.0	1.1340e-01	9.6886e-01
$[\sin(x)] - 0.3$	-4.440892098500626e-16	-5.551115123125784e-16	1.1e+15	1.7643e+14	6.7554e+14
$[\sin(x) - 0.3] * 10$	-4.440892098500626e-15	-5.551115123125784e-15	1.4e+15	2.2053e+14	1.0000e+00

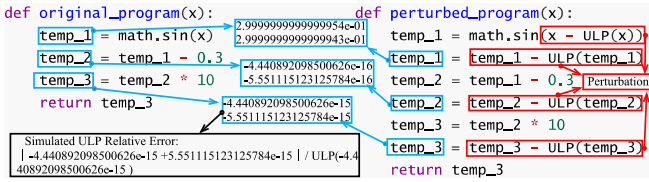


Fig. 4. An example of our error method of simulating floating-point errors. The intermediate results of the original program and the perturbed program are presented.

perturbation into `temp_1` (cf. Fig. 4), the difference between two results of the operation is substantial because of the amplification from the large condition number, which is consistent with the oracle error magnitude. Finally, while the condition number of the third operation is small, the large error from the second operation is propagated to the final result. Therefore, our method is capable of simulating errors and comprehensively modeling their propagation dynamics. Note that Fig. 4 demonstrates the perturbation process via direct source code manipulation. In contrast, our method employs an LLVM pass-based perturbation mechanism, treating the source code as a black box.

Our simulation of floating-point errors are reliable. In the ATOMU experiments, a relative error threshold of 0.001 was adopted to classify errors as significant (≥ 0.001) or nonsignificant (< 0.001). Their published results comprise 878 cases, with 800 nonsignificant and 78 significant errors. We use these results as ground-truth validation data to evaluate our error simulation approach. Our experiments reveal a false negative rate of 0.23% (2 cases) and a false positive rate of 1.37% (12 cases). Moreover, among our false positives, the smallest error

measures 2.38×10^{-5} , demonstrating the reliability of ICE's error-simulation methodology.

Our perturbation approach is efficient. For GSL functions, the perturbed version incurs a runtime overhead of approximately $4.5\times$ compared to the original program, while for the HSED dataset, the runtime overhead is about $2\times$. These results indicate that our perturbation strategy introduces only modest time overhead. In particular, the overhead is negligible when compared with high-precision implementations, which often require more than two orders of magnitude longer execution time than the original programs [22].

B. Approach of ICE

In this subsection, we present the detailed approach of ICE. First, we transform the existing program into a perturbed program that automatically contains injected errors. Second, we select fewer input values from the entire input domain and calculate the error in the output based on comparing the output from the original and the perturbed program. Finally, we connect the inputs that correspond to larger output errors as a new search space for the existing tools.

Step 1: Program transformation for statically injecting perturbation. In the first step, we simulate floating-point errors by directly injecting errors into the original program to facilitate the search of input values that may lead to a large error in the output of the program. Such injection can be done either statically on the program or dynamically during the execution of the program. Due to the fact that dynamic approach contains large overhead and would be time consuming, it would not be a practical choice when we have a large number of input values to be evaluated for the program. Therefore, we opt to design a static program transformation approach that inject such errors into

Algorithm 1 The Details of Injection Implementation

```

1: Input: Floating-point program  $P$ , input  $x$ 
2: Output: Perturbed output  $y$ 
3:  $x' \leftarrow x - \text{ULP}(x)$ 
4: Set  $x'$  as the input of  $P$ 
5: for each floating-point operation  $op$  in  $P$  do
6:    $r \leftarrow \text{Execute}(op)$ 
7:    $r \leftarrow \text{Perturb\_function}(r)$ 
8:   Replace the result of  $op$  with  $r$ 
9: end for
10:  $y \leftarrow \text{output of } P$ 
11: Function  $\text{Perturb\_function}(r)$ 
12:   return  $r - \text{ULP}(r)$ 
13: return  $y$ 

```

the program itself. Specifically, we inject small perturbations into floating-point programs based on the LLVM pass. First, we utilize Clang 18³ to compile given floating-point programs into LLVM Intermediate Representation (IR) [29]. Second, our LLVM pass scans the LLVM IR instruction by instruction. When a floating-point atomic operation is encountered, we insert a function call that subtracts one ULP from the result of each operation as shown in Algorithm 1. Since elementary arithmetic operations tend to be bounded by an upper limit [8], i.e., $1 + 1/1000$ ULPs, we set the perturbation magnitude to one ULP. We calculate the ULP of the result of the operation. Subsequently, we subtract one ULP from the result and return the new operation result for the following operations. Taking the first operation of the instance in Section IV-A as an example, the line $\text{temp_1} = \text{math.sin}(x)$ would be transformed into $\text{temp_1} = \text{math.sin}(x - \text{ULP}(x)) - \text{ULP}(\text{math.sin}(x - \text{ULP}(x)))$.

After the transformation, we have two versions of the floating-point program, including 1) the original version of the program and 2) a new version of the program where perturbations are automatically done during execution.

Step 2: Computing floating-point errors by using all grid points as input arguments. Floating-point programs contain a huge number of possible input arguments; while only a small fraction of them lead to large errors (cf. Section III). The input arguments of the given floating-point programs are typically double-precision floating-point numbers, and calculating all corresponding errors is impractical due to the computational cost. Hence, we reduce the inputs to identify input domains of a given floating-point program where substantial errors occur, with the following two sub-steps.

Step 2.1: We construct a detection grid to reduce possible input arguments. The granularity of the grid is contingent upon the availability of computational resources and temporal constraints. Taking half-precision (e.g., 16 bit) floating-point numbers as grid points as an example, since a typical floating-point number is with double-precision (i.e., 2^{64} possible inputs), using half-precision floating-point numbers as grid points would reduce the total possible inputs to the $1/2^{48}$ of the original

amount of possible inputs. The choice of using grid points to reduce possible inputs is motivated by two key considerations. First, as two inputs with proximate values tend to yield similar condition numbers, numbers near the input argument which results in large errors tend to lead to big errors (cf. Fig. 2). Second, floating-point numbers are non-uniformly distributed, and lower-precision (e.g., half-precision) floating-point numbers exhibit a distribution similar to that of double-precision floating-point numbers. Therefore, the distribution of our grid points aligns with the distribution of potential inputs to the given floating-point program.

Step 2.2: We obtain the simulated errors by taking grid points as inputs to execute the programs. We execute the original programs and the perturbed versions by taking the values of our grid points as input arguments. Subsequently, we calculate the differences between results of the two programs, which are considered as our simulated errors.

Why not use high-precision program to estimate errors at the grid points? An alternative approach in this step is to use a high-precision program to calculate the output and compare with the original program, in order to estimate the error. However, this traditional approach contains two drawbacks: long execution time and instability. Our perturbation-based approach is designed to mitigate these two issues. First, since we do not use high-precision programs, the computational time required by our approach for error estimation is approximately only five times that of the original program. Second, since our approach is grounded in solid theoretical foundations, it effectively addresses the instability issue. For instance, the blue points, derived from ICE, consistently indicate large errors in the vicinity of the peak in Fig. 2. This demonstrates that ICE can reliably identify erroneous regions by detecting one input with large error.

Step 3: Connecting grid points with large errors for generating erroneous domains. In the final step of the approach of ICE, we need to generate input domains that are more likely to be erroneous to reduce the search space of existing tools (e.g., ATOMU [8], HSED [19], and FPCC [23]).

We select the top $n\%$ of error-prone grid points from the last step to construct the erroneous domains. It is worth noting that the parameter n is designed as a flexible variable to account for the distinct error distribution characteristics inherent to different floating-point programs. Instead of directly selecting the top $n\%$ of points as endpoints, we extend these points by one additional point. These extended points are then connected to form erroneous domains. Fig. 5 illustrates three such ranges, with the pentagrams representing the extended points. All the points above the threshold are top $n\%$ grid points. This extension serves two main purposes. First, ICE aims to provide conservative domains for tools that identify error-inducing inputs, so we cannot ignore the lines where certain parts exceed the threshold. Second, certain programs may be undefined at specific points, such as 0, where developers might assign a constant value. In such cases, ICE could not detect errors at these points due to the lack of operations. However, surrounding points are often prone to possess substantial errors. Algorithm 2 illustrates the process of constructing erroneous domains. In Algorithm 2,

³<https://clang.llvm.org/>

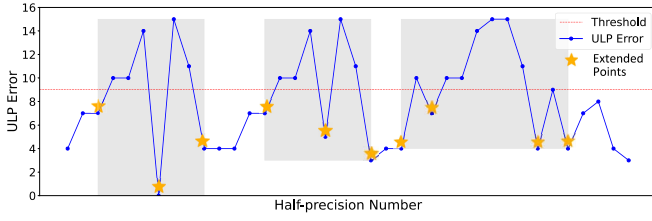


Fig. 5. An illustration of the domain generation process. We expand the neighboring grid points and combined adjacent intervals into single units.

Algorithm 2 Build Erroneous Domains by Top $n\%$ Grid Points

Require: The index of top $n\%$ grid points among the half-precision floating-point numbers $S = \{s_1, s_2, \dots, s_m\}$ and a list of half-precision floating-point numbers $X = \{x_1, x_2, \dots, x_n\}$.

Ensure: A set of ranges $R = \{[a_1, b_1], [a_2, b_2], \dots, [a_k, b_k]\}$

```

1: Initialize an empty set  $Y \leftarrow \emptyset$ 
2: for each point  $s_i \in S$  do
3:   if  $s_i - 1$  is not in set  $S$  and  $X[s_i - 1]$  exists then
4:     Add  $s_i - 1$  to the set  $Y$ 
5:   end if
6:   if  $s_i + 1$  is not in set  $S$  and  $X[s_i + 1]$  exists then
7:     Add  $s_i + 1$  to the set  $Y$ 
8:   end if
9: end for
10: Initialize an empty list  $R \leftarrow \emptyset$ 
11:  $start \leftarrow y_1$ 
12: for each point  $y_i \in Y$  do
13:   if  $y_{i+1}$  exists and  $y_{i+1} \neq y_i + 1$  then
14:     Add the range  $[X[start], X[y_i]]$  to the list  $R$ 
15:      $start \leftarrow y_{i+1}$ 
16:   end if
17: end for
18: Add the final range  $[X[start], X[y_{last}]]$  to the list  $R$ 
19: return  $R$ 

```

lines 2–9 extract the extended points and store them in Y , which serve as endpoint candidates for constructing the erroneous domains. Lines 12–17 then use these extended points to form the erroneous domains and merge the consecutive intervals. After this step, ICE would generate erroneous domains, which can be given to existing tools (e.g., ATOMU [8], HSED [19], and FPCC [23]) to facilitate their automated identification of inputs that lead to large float-point errors. In the next section, we present our evaluation of using these tools together with ICE to improve their detection.

V. EVALUATION

In this section, we conduct a comprehensive evaluation to assess ICE. It is worth noting that **the goal of ICE is not to replace existing approaches, but rather complement existing approaches**. Therefore, we would like to study whether our proposed method, ICE, can enhance the performance of existing error-inducing input detection tools.

We begin our evaluation by introducing the experimental setup, including the selected three state-of-the-art tools, ATOMU [8], HSED [19], and FPCC [23] - along with the datasets employed for assessment. Subsequently, we select the parameter and present the experimental results, where we demonstrate the potential of our method to enhance the performance of error-inducing input search tools from two perspectives: identifying more error-inducing inputs and shortening the search time.

A. Evaluation Setup

1) *State-of-the-art Tools*: To evaluate the performance of our proposed method, we apply the domains generated by ICE to three state-of-the-art tools: ATOMU [8], HSED [19], and FPCC [23].

ATOMU is the first tool capable of identifying error-inducing inputs without the need for high-precision programs to generate oracles. It uses an evolutionary algorithm [30] to explore the input domain—specifically, the entire double floating-point number domain—and find inputs that result in large condition numbers (see Section II-C) for individual atomic operations. Subsequently, ATOMU ranks the detected cases and outputs an uncertain number of results.

HSED employs a hierarchical approach to search across different precision levels. 1) **Initial Layer**: HSED evaluates all half-precision floating-point numbers to identify the largest one. Then the number is forwarded to the subsequent layer for further analysis. 2) **Second Layer**: If the input from the initial layer exceeds a threshold of 100 ULPs, HSED expands the input to single-precision floating-point numbers by extending the significand. Then, HSED computes their errors and the one with the largest error is forwarded to the final layer for further processing. Conversely, if the error of the input is minor, HSED proceeds with a random search to produce the case with the largest error to the final layer. 3) **Final Layer**: HSED similarly extends this single-precision number into the double-precision space. However, due to time constraints, exhaustively traversing all double-precision numbers is infeasible, so a random sampling approach is employed, with the highest-error number taken as the final result.

FPCC stands as the most recently developed tool, which initially segregates the search space. Subsequently, FPCC performs direct search and line search for each partition. Notably, FPCC employs chain conditions as loss functions to enhance search efficiency, where chain conditions refer to the condition numbers of the entire programs.

2) *Experimental Environment*: Our experiments are conducted within a Docker container running Ubuntu 24.04 as the operating system. We utilize a machine equipped with an AMD Ryzen 5 9600X @ 3.90 GHz CPU and 32GB of RAM.

3) *Parameter Selection*: In our method, we utilize the top $n\%$ of error-prone half-precision floating-point numbers as endpoints to generate erroneous domains. Given that n is a tunable parameter, we should choose a reasonable one. Based on the approximately 320 ranges, FPCC identifies an average of 12 error-inducing inputs in GSL functions, corresponding to an

error rate of 3.75%. Given that ICE is a conservative method, in our experiment, we set n to 5. Following the work of ATOMU [8], we define a **relative error exceeding 0.001 as a significant error**, which serves as a key metric for evaluating the effectiveness of our approach.

B. Evaluation Results

In this subsection, we present the experimental results of our evaluation of the proposed method, ICE, structured around three research questions (RQs). To investigate whether ICE can assist downstream tools in discovering more error-inducing inputs, we formulate RQ1. To examine whether ICE can reduce the running time of downstream tools, we introduce RQ2. To assess the reliability of ICE's error-estimation mechanism, we pose RQ3. For each RQ, we outline the motivation and methodology before discussing the corresponding results.

RQ1 (general evaluation of ICE): Can ICE help tools identify more error-inducing inputs?

Motivation. Existing tools, such as ATOMU and HSED, aim to identify error-inducing inputs but often rely on an overly broad and sparse search space, where only a limited number of inputs lead to significant errors. Therefore, in this research question, we aim to evaluate whether ICE's capacity to narrow the search domain can enhance the detection.

Approach. In this section, to analyze the impact of these new domains produced by ICE on identifying error-inducing inputs, we use the reduced domains as the search space for ATOMU, HSED, and FPCC, instead of the original, broader (entire) domains of floating-point numbers. We consider errors with a relative error greater than 0.001 as significant errors (cf. Section V-A). Since ICE aims to work together with existing approaches, to answer this RQ, we design the evaluation of ICE always when it works with existing approaches.

Evaluation with ATOMU. To ensure a fair evaluation, we follow the settings of prior research [8] by using the same evaluation dataset, which comprises 88 functions selected from the GNU Scientific Library (GSL). We adhere to the parameters and random seeds provided in ATOMU's open-source packages. Note that as pointed out in previous research [19], using the entire double floating-point number domain as the input domain is excessively large and lacks practicality. Therefore, in our experiments, we constrain the input domain to the range of half-precision floating-point numbers, specifically within the interval $[-65504, 65504]$.

Evaluation with HSED. The dataset of HSED consists of 32 mathematical expressions and their corresponding input domains. We identify and discard a duplicate entry. In the authors' experiments, HSED operates within relatively smaller domains, such as $[0.01, 100]$. However, these ranges may fail to capture certain error-inducing inputs. For example, the input range and expression of the function NMSEexample37 are $[0.01, 100]$ and $\exp(x) - 1.0$, respectively. According to the Equation (6), if the input is very close to zero, this function's result will exhibit a significant error. However, with a minimum input value of only 0.01, the specified range does not trigger the bug, thereby limiting HSED's detection capabilities. Therefore, we modify

all function ranges to $[-100, 100]$. The outcomes of its initial layer can be categorized into two scenarios (cf. Section V-A). In the first scenario, if the initial output does not exceed 100 ULPs, HSED performs a random search within the initial domain, where ICE may enhance HSED by identifying domains with larger errors. In the second scenario, where the error of the initial output is significant, ICE would yield the same result, as the first layer exhaustively examines all half-precision floating-point numbers (with the reduced space by ICE already included in this domain) to find the number with the largest error. In this case, ICE merely reduces the traversal space and potentially leads to reduced search time (cf. RQ2). Therefore, ICE can only result in larger errors for HSED in the first scenario. If a specific input region is illegal (e.g., negative inputs), the function returns non-finite values (e.g., NaN). Consequently, the grid points of ICE in this region cannot yield valid error measurements. Since ICE constructs domains based on high error values, it automatically excludes these illegal regions from the generated erroneous domains. Therefore, uniformly setting the search interval to $[-100, 100]$ does not negatively impact the performance of ICE.

Evaluation with FPCC. FPCC utilizes the same datasets as ATOMU, specifically 88 GSL functions. The first step of FPCC is splitting the entire space into some ranges, where each range produces a single result. To assess whether ICE can enhance FPCC, we adopt all half-precision floating-point numbers as the endpoints for partitioning, resulting in 63,485 small ranges, rather than employing the original partition of FPCC. Given that the outputs of ICE consist of erroneous domains, we segregate them using the same half-precision floating-point endpoints. Subsequently, we evaluate whether FPCC can detect comparable error-inducing inputs by using the downsized ranges as those identified using the entire 63,485 ranges. Since FPCC generates one result for each range, ICE is not able to help produce more error-inducing inputs. We evaluate whether FPCC can detect a comparable number of error-inducing inputs when using our erroneous domains. The closer the quantity, the stronger the indication of the high quality of our erroneous domains.

Results. ICE has shown to be effective in assisting ATOMU by identifying additional error-inducing inputs. As shown in Table II, ATOMU initially detected 31 error-inducing inputs from the original domain. With the application of our method, this number increases to 60, representing a growth factor of approximately 93.54%. Additionally, while ATOMU initially identifies 16 functions containing significant errors, the application of ICE results in the detection of 36 functions, reflecting a growth of 125%. We would like to note that all the detected errors are verified true positive errors. Consequently, developers are able to address more than twice the number of programs containing significant errors. We believe these condensed domains enable ATOMU to detect subtle errors more effectively. For the function in Fig. 1 (cf. Section III), ATOMU fails to detect any error-inducing inputs when operating within the original search space. However, when employing the domains of ICE, ATOMU successfully identifies a significant error at the specific point 0.5773502691896256, highlighting the critical

TABLE II
NUMBER OF ERROR-INDUCING INPUTS DETECTED BY ATOMU BASED ON
ORIGINAL DOMAINS AND OUR ERRONEOUS DOMAINS

Function Index	# Err. Inp. by only ATOMU	# Err. Inp. based on ICE	Function Index	# Err. Inp. by only ATOMU	# Err. Inp. based on ICE
0	1	1	54	0	1
2	0	1	55	0	1
3	0	1	56	0	1
4	1	1	58	0	1
5	1	1	59	0	1
6	1	1	61	0	1
7	1	1	64	1	2
8	0	2	73	0	1
9	2	2	80	0	1
10	0	2	81	0	1
11	1	2	83	0	1
21	1	1	88	2	2
22	1	1	90	3	3
24	0	1	101	1	2
27	0	1	102	0	1
32	0	1	104	6	4
40	0	1	105	5	10
49	0	1	106	3	4

Note 1: Results for a function are not provided if no error-inducing inputs are observed.
Note 2: We abbreviate "Err. Inp." to "Error-inducing Inputs".

importance of search space optimization in floating-point error detection. Given that different inputs may trigger the same bug, we adopted the criterion based on triggered instructions with large condition numbers to further evaluate the performance of ICE: if two inputs lead to significant errors due to the same instruction, they are considered to trigger the same bug. Our experimental analysis shows that after deduplication, ATOMU with ICE was able to detect 30 additional instructions, while only two instructions were missed compared to the pure ATOMU. The two missed bugs were likely attributable to the inherent randomness in ATOMU's algorithm. These results indicate that ICE enhances the effectiveness of ATOMU.

HSED and FPCC demonstrate comparable performance in detecting error-inducing inputs with reduced erroneous domains. HSED detects 12 error-inducing inputs based on both the original search space and our erroneous domains. Based on the entire ranges, FPCC finds 14,943 error-inducing inputs in 34 functions. Within our erroneous domains, FPCC detects 8,574 error-inducing inputs while still identifying all 34 error-prone functions. For function 101, FPCC identifies 10,238 error-inducing inputs when considering all ranges. However, since the total number of ranges is 7,258, it is hard to cover all ranges using only the top 5% grid points. Despite this limitation, our method successfully detects 4,560 error-inducing inputs, which is a highly satisfactory outcome. For 30 out of the 34 functions, we detect the same number of error-inducing inputs. Furthermore, employing the same criterion as ATOMU, FPCC with ICE successfully identified six additional instructions while missing only two instructions. Such detection results are achieved by a drastically reduced input ranges from 63,485 to 10,097.48 per function, demonstrating ICE can help remain the valuable ranges.

ICE significantly boosts ATOMU by identifying more erroneous functions and additional error-inducing inputs, with reduced domains. HSED and FPCC maintain comparable effectiveness with drastically reduced domain.

RQ2 (addressing limitation 1): Can ICE improve the time efficiency of existing tools without adding overhead?

Motivation. In RQ1, we demonstrate that ICE effectively enhances existing tools for better error detection. However, a potential concern is that ICE's additional steps to narrow the search domain might introduce excessive overhead. Additionally, we want to determine if the reduced search space can actually improve the time efficiency of these tools. Thus, this research question investigates whether ICE can enhance the time efficiency of existing tools without incurring overwhelming overhead.

Approach. To address this research question, we measure the time taken to generate the reduced range and compare the overall time consumption of existing tools with and without the use of ICE.

Results. ICE can reduce the runtime of the first layer of HSED without adding overhead. As shown in Table III, the average runtime per function of the first layer is reduced from 3.4192 seconds to 0.5926 seconds, which is only 17.33% of the original runtime. For example, the function test05_nonlin1_test2 sees its runtime decrease from 1.4745 seconds to 0.0034 seconds, a reduction of 99.77%. Additionally, the average execution time per function of ICE is just 0.0204 seconds, which is negligible compared to the overall runtime of HSED. Overall, our method effectively enhances HSED's efficiency.

ICE significantly reduces the runtime of FPCC. Given that FPCC and ATOMU utilize the same datasets, the average execution time per function of ICE is 0.1317 seconds. In comparison, the runtime of FPCC per function with and without our erroneous domains is 0.3257 and 14.9199 seconds, respectively, demonstrating a substantial reduction of 97.82%.

ICE does not drastically increase the execution time for ATOMU. Since ATOMU is based on a genetic algorithm to search and being an already efficient approach [19], we do not expect the execution time to be reduced with ICE. Instead, we would like to evaluate the overhead that ICE gives to ATOMU. We find that the average execution time per function of ICE is 0.1317 seconds, while the average execution times of ATOMU with and without our erroneous domains are 0.2478 and 0.1578 seconds, respectively. To further understand the reason for the extra 0.1 second overhead per function, we check each detection result and find that the extra time is due to being able to identify larger errors. With the erroneous domains provided by ICE, bigger errors are more possible to be detected, leading to ATOMU, a genetic algorithm based approach to run for more iterations to identify the bigger error.

The application of ICE significantly reduces the runtime of HSED's first layer by 82.67%. Our erroneous domains achieve a substantial reduction in the execution time of FPCC by 97.82%. ICE slightly increases the execution time of ATOMU due to the search of bigger errors.

TABLE III
RUNTIME OF THE FIRST LAYER OF HSED WITH AND WITHOUT USING ICE

Name	Original Runtime	Runtime After Deploying ICE	Time Required by ICE	Name	Original Runtime	Runtime After Deploying ICE	Time Required by ICE
sqrt	3.250349764	0.00412063	0.0300964	NMSEexample38	11.56483583	0.023319899	0.0207582
sqrt_add	2.175041779	0.005919716	0.0175729	NMSEproblem334	2.848812373	0.005988927	0.0188074
exp1x	2.905856223	0.01373047	0.0178714	NMSEproblem333	2.128904479	2.082105828	0.0221916
exp1x_log	7.085062856	0.021545504	0.0182807	NMSEproblem331	1.64842244	1.626241357	0.0198584
NMSEexample37	2.525268319	0.014384043	0.0172084	NMSEexample36	2.155274816	2.10257538	0.0198586
NMSEproblem336	11.04337189	0.033721944	0.0185184	NMSEexample35	4.351709017	3.78526124	0.0209652
NMSEexample39	1.412794263	0.685963327	0.0193067	NMSEexample34	1.255397915	0.010923788	0.0186799
NMSEproblem341	1.227433244	0.006952811	0.0184981	NMSEexample31	2.019416419	0.00395351	0.0184518
NMSEsection311	3.348448735	3.040888582	0.0183398	test05_nonlin1_r4	1.780041892	0.004719329	0.0188489
NMSEproblem345	2.371113561	0.020322401	0.0189205	test05_nonlin1_test2	1.47452076	0.00337066	0.0173326
NMSEproblem337	3.454959642	2.940428973	0.0177779	intro_example_mixed	1.523967039	0.001825211	0.0177827
verhulst	1.886198697	0.002310507	0.0189187	sineOrder3	2.081014337	0.002561863	0.019921
predatorPrey	2.176160609	0.003794595	0.0217597	bsplines3	1.552507127	0.694058746	0.0183872
logexp	3.245993139	0.037222149	0.0176413	NMSEexample310	7.190996073	6.206762992	0.021132
sine	3.935458866	0.005675382	0.032671	NMSEproblem343	5.015673578	2.261526816	0.0182043
carbonGas	2.663839777	2.644590667	0.0228151	Average	3.41920946	0.5925982313	0.0203873125

RQ3 (addressing limitation 2): Are high-precision floating-point programs better alternatives of estimating errors?

Motivation. In RQ1 and RQ2, we demonstrate that ICE can effectively and efficiently enhance the performance of existing error-inducing input detection tools. While most prior studies rely on oracles derived from high-precision programs to compute floating-point errors, this approach suffers from two significant limitations: prolonged execution time and instability (cf. Section III). In particular, the instability of errors would be detrimentally harmful to ICE, because important regions may be ignored. To address the limitation of using high-precision programs, we propose a perturbation injection-based approach for error estimation. Therefore, in RQ3, we would like to assess whether simply using high-precision programs is actually a better alternative.

Approach. To address this research question, we use the high-precision-based approach to calculate errors of grid points to perform ICE. First, we assess whether the high-precision program-based approach takes more time. Subsequently, to assess whether our approach can solve the instability issue, we compare the performance of ICE when using errors derived from the two approaches.

Result. Our approach significantly reduces runtime compared to the high-precision program-based method. The high-precision program-based approach requires 11.3992 seconds per function for ICE to construct erroneous domains, whereas our approach completes the task in only 0.0204 seconds, representing a mere 0.18% of the former's runtime. Similarly, for the datasets of ATOMU and FPCC, the high-precision program-based approach consumes 71.1822 seconds per function, while our approach requires only 0.1317 seconds. These results highlight the significantly prolonged execution time of the high-precision program-based approach.

Using high-precision programs to estimate errors would lead to fewer detected error-inducing inputs compared to our current approach of error simulation. When utilizing ICE with our estimated errors, ATOMU detects 60 error-inducing inputs across 36 functions. In contrast, the

high-precision program-based approach enables ATOMU to identify only 46 error-inducing inputs in 29 functions. For HSED, the high-precision approach detects merely three error-inducing inputs, whereas our approach identifies 12. Similarly, FPCC detects 4,229 error-inducing inputs across 30 functions using ICE with high-precision errors, while our method identifies 8,574 error-inducing inputs in 34 functions. These results demonstrate the improved effectiveness of our approach in constructing erroneous domains.

High-precision program-based approach is not a better alternative. Compared to our perturbation injection-based approach, it requires about 500 times more runtime to construct ICE's erroneous domains. Moreover, the domains generated demonstrate significantly lower performance.

VI. DISCUSSION

In this section, we discuss whether the erroneous domains generated by ICE may miss many errors. We take multi-input functions as our validation dataset and conduct a series of ablation studies. Since HSED is a deterministic tool, as long as an interval contains error-inducing inputs, HSED is able to detect the same inputs consistently. In contrast, FPCC and ATOMU are non-deterministic; each run may identify different error-inducing inputs. Moreover, different error-inducing inputs may still trigger the same underlying bugs. To better evaluate and compare the search outcomes of existing tools, we identify, for each function, the high-condition-number instructions triggered by all discovered error-inducing inputs as the sources of bugs, and record the deduplicated count of such instructions. As the erroneous domains produced by ICE cover all error-inducing inputs identified in the original domains in all HSED-based experiments in this section, we focus our discussion on FPCC and ATOMU.

TABLE IV
RESULTS OF THE FOUR ABLATION STUDIES. #FUNCTION DENOTES THE NUMBER OF FUNCTIONS IN WHICH ERROR-INDUCING INPUTS WERE DETECTED.
#INSTRUCTION CORRESPONDS TO THE NUMBER OF INSTRUCTIONS TRIGGERED BY THESE INPUTS

Perturbation Magnitude																					
ATOMU											FPCC										
Magnitude (ULP)	5	4	3	2	1	-1	-2	-3	-4	-5	Magnitude (ULP)	5	4	3	2	1	-1	-2	-3	-4	-5
#Instruction	45	46	51	45	52	53	47	46	51	50	#Instruction	54	55	51	58	55	56	53	53	53	53
#Function	33	32	34	32	35	36	33	34	32	35	#Function	34	34	34	35	34	34	34	34	34	34
Randomized Perturbations																					
ATOMU											FPCC										
Trial Index	1	2	3	4	5	6	7	8	9	10	Trial Index	1	2	3	4	5	6	7	8	9	10
#Instruction	48	40	39	45	41	43	40	40	42	42	#Instruction	52	52	53	52	47	52	55	51	55	52
#Function	32	30	29	30	31	30	29	31	30	31	#Function	33	32	34	33	31	32	33	31	33	32
Different values of n																					
ATOMU											FPCC										
n%	0.10%	0.25%	0.50%	1%	2%	3%	4%	5%	7.50%	10%	n%	0.10%	0.25%	0.50%	1%	2%	3%	4%	5%	7.50%	10%
#Instruction	43	48	43	48	43	45	41	53	48	35	#Instruction	54	52	57	56	54	56	55	56	59	36
#Function	31	33	33	33	30	33	32	36	32	27	#Function	34	32	35	35	34	35	34	34	34	25
Grid Granularity																					
ATOMU											FPCC										
#Mask Bit	0	1	2	3	4	5	6	7	8	9	#Mask Bit	0	1	2	3	4	5	6	7	8	9
#Instruction	53	42	47	46	51	49	39	39	37	30	#Instruction	56	56	55	56	56	55	54	51	44	37
#Function	36	31	32	32	33	32	29	29	29	24	#Function	34	34	34	34	34	34	32	31	32	27

To verify whether the erroneous domains generated by ICE would miss a substantial number of errors, we expanded the neighborhoods of the ICE-generated domains and tested them with FPCC and ATOMU to search for additional error-inducing inputs. Specifically, each small interval produced by ICE was extended by 50% both to the left and to the right, resulting in new domains with the same total length as the erroneous domains originally identified by ICE. The results show ATOMU and FPCC detected error-inducing inputs in the expanded intervals for only 10 and 12 functions, respectively, compared to 36 and 34 functions found within ICE's original erroneous domains. The instructions triggered by FPCC in the expanded domains form a subset of those already captured within ICE's erroneous domains. Meanwhile, ATOMU triggered only one additional new instruction. These findings indicate that the expanded domains contain very few additional errors, demonstrating that the erroneous domains identified by ICE already encompass the vast majority of errors.

To better evaluate our method, we conduct a series of ablation studies, the results of which are summarized in Table IV. We first investigate the impact of perturbation magnitude by varying it from +5 ULP to -5 ULP. The experimental results show that the performance differences across different perturbation magnitudes are relatively small; among them, the -1 ULP perturbation selected by ICE yields the best overall performance. We then evaluate randomized perturbations. The results indicate that randomized perturbations perform worse than constant perturbations. We attribute this degradation to the fact that randomized perturbations lead to inconsistent error values across grid points, which in turn negatively affects the quality of the generated erroneous domains. Regarding the choice of n , we observe that, for ATOMU, setting n to 5% achieves the best performance. For FPCC, the performance remains largely stable until n reaches 7.5%, beyond which a noticeable degradation is observed. Finally, we study the effect of grid granularity.

Initially, we use float16 values as grid points. To obtain different grid granularities, we progressively truncate the least significant bits of the mantissa, thereby increasing the spacing between grid points. The results show that overly coarse grid densities lead to degraded performance.

To further evaluate the generality and robustness of our method, we extend our evaluation to more complex scenarios by considering multi-input functions. However, native support for multi-input functions is limited: ATOMU lacks multi-input support entirely, and while FPCC supports multi-input programs, it is incompatible with third-party libraries (e.g. GSL functions). To overcome this, we leveraged the fact that our selected functions take two inputs, one of which is an integer. By enumerating all possible integer inputs and decomposing the two-input function into some single-input functions (with real parameters preserved), we enabled ATOMU and FPCC to validate ICE's performance on multi-input functions. In total, GSL provides 23 such functions. In this study, we initially attempted to include all of them to ensure comprehensive coverage. However, during inspection, we found that for seven functions, the corresponding high-precision implementations are time-consuming and problematic, making them unsuitable as ground-truth references. Therefore, we exclude these cases and conduct experiments on the remaining 16 functions. We then employed ATOMU and FPCC as downstream evaluators to assess the effectiveness of the erroneous domains generated by ICE. Due to the extensive computation required, experiments were conducted using multi-core parallelization; consequently, all reported times represent single-CPU execution time. Table V demonstrates significant improvements in both detection capability and efficiency. For ATOMU, the tool identified only six instructions using the original domains but successfully detected 16 instructions (including the original six) on the erroneous domains generated by ICE. In terms of efficiency, ICE generation required 1.53 days. ATOMU took 10.58 days on the original domains, compared to 7.45 days on the ICE-generated domains. For FPCC, while

TABLE V
EVALUATION OF DETECTION CAPABILITY AND EFFICIENCY FOR ATOMU AND FPCC ACROSS ORIGINAL AND ICE-GENERATED DOMAINS ON MULTI-INPUT FUNCTIONS

Function Name	ATOMU				FPCC			
	Number of Instructions		Time(s)		Number of Instructions		Time(s)	
	Original Domains	ICE's Domains	Original Domains	ICE's Domains	Original Domains	ICE's Domains	Original Domains	ICE's Domains
gsl_sf_bessel_Jn_e	0	0	89,436	64,182	0	0	2,579,442	78,127
gsl_sf_bessel_Yn_e	0	0	121,588	86,375	0	0	3,146,650	102,438
gsl_sf_bessel_In_e	0	0	23,877	21,194	0	0	1,098,456	33,847
gsl_sf_bessel_Kn_e	0	0	5,962	4,188	0	0	187,533	6,281
gsl_sf_bessel_jl_e	0	5	227,326	170,622	4	4	5,544,523	193,613
gsl_sf_bessel_yl_e	2	2	168,443	118,236	5	5	3,366,439	95,982
gsl_sf_legendre_Pl_e	0	0	10,363	2,218	0	0	73,674	2,448
gsl_sf_legendre_Ql_e	0	0	3,719	846	0	0	33,833	953
gsl_sf_hermite_phys_e	0	5	152,987	75,126	2	2	1,310,191	42,874
gsl_sf_hermite_prob_e	4	4	64,215	56,432	3	3	930,260	29,127
gsl_sf_bessel_In_scaled_e	0	0	16,505	17,972	0	0	836,272	25,971
gsl_sf_bessel_Kn_scaled_e	0	0	4,235	2,930	0	0	132,971	4,573
gsl_sf_bessel_il_scaled_e	0	0	10,269	7,996	0	0	466,516	12,743
gsl_sf_bessel_kl_scaled_e	0	0	2,175	1,222	0	0	51,605	1,780
gsl_sf_psi_n_e	0	0	765	202	0	0	23,638	724
gsl_sf_taylorcoeff_e	0	0	12,674	14,082	0	0	245,455	7,194
SUM	6	16	10.58 days	7.45 days	14	14	231.80 days	7.39 days

it identified the same 14 instructions across both domains, the efficiency gain was substantial: FPCC required 231.80 days on the original domains but only 7.39 days on the ICE-generated domains. In conclusion, these results confirm that ICE is highly effective even when applied to complex, multi-input programs.

VII. THREATS TO VALIDITY

In this section, we discuss the threats to the validity of this work.

External validity. We only apply ICE to three existing error-inducing input detection tools and the enhancement may not generalize to other tools. However, our method aims to provide reduced, higher-quality domains for existing tools, which will undoubtedly enhance their effectiveness. In addition, since ICE does not use high-precision programs to compute the oracle, the time it requires is negligible compared to existing tools, aside from ATOMU. The significant differences in the algorithms of these three tools further demonstrate the generality of our approach. Our experiments are conducted primarily on stateful functions with known state variables. The generalizability of ICE to stateful functions with unknown state variables remains a limitation. When external variable values are unknown, our method may fail due to the inability to construct deterministic grids. Nevertheless, in such scenarios, ICE's downstream methods (e.g., FPCC) likewise struggle, as the output associated with a specific input and the corresponding relative error are no longer deterministic.

Internal validity. We select the top 5% of half-precision floating-point numbers to generate erroneous domains. Although we conduct experiments to evaluate this selection, we do not explore different parameter settings for varying datasets.

An excessive number of endpoints may result in trivial reductions, while too few endpoints could lead to the omission of critical domains. Nevertheless, the experimental results are highly promising, demonstrating the strong adaptability of our proposed method.

A potential threat lies in whether the perturbation operation itself introduces computational noise that distorts the error estimation. We believe that this threat is minimal for two reasons. First, the perturbation operation (e.g., $x - \text{ULP}(x)$) is performed using exact floating-point arithmetic under IEEE 754 semantics. By definition, $\text{ULP}(x)$ represents the exact gap between x and its successor [24]. Consequently, the difference results in a strictly representable floating-point value (conceptually, the predecessor of x) without incurring additional rounding errors. Second, even in the hypothetical case of error overestimation, the effectiveness of ICE remains robust. This is because ICE calculates the error at grid points and selects those with larger error values as interval endpoints. Therefore, overestimation of error does not disrupt the construction of these intervals and the performance of ICE.

Construct validity. We directly use the simulated errors as real errors, which may contain some faults. However, the simulated errors are grounded in solid theory. Furthermore, RQ3 demonstrates that our simulated error performs well and can mitigate two problems of the high-precision program-based approach.

VIII. RELATED WORK

In this section, we discuss some closely related areas of our work.

Detecting errors in floating-point operations. Valgrind [31] is a dynamic binary instrumentation tool that could build heavyweight dynamic binary analysis tools. Valgrind offers support for a technique shadow values, which pairs each register and

memory value with an additional value that describes it. Based on this tool, many methods have been proposed, including FPDebug [32], Verrou [33] and Herbgrind [34]. FPDebug utilizes random tests to find catastrophic cancellations and rounding errors founded on shadow variables in higher precision. Verrou uses the Monte Carlo algorithm to find bugs. Herbgrind is a tool designed to identify the sources of errors in large-scale programs. It works by randomly sampling and computing values with high-precision using the MPFR library. Herbie [35] utilizes heuristic search to localize rounding errors and could also optimize programs by rewriting expressions in error-prone parts. Bao et al. [7] proposed an on-the-fly efficient monitoring technique that could track inflation bit propagation. If one inflation bit reaches a critical execution point, such as a conditional statement, where the inflated error could cause a significant execution difference, instability will be reported. RAIVE [36] reduces the overhead of their approach by adding some decisions. Chowdhary et al. [37] proposed a novel prototype for parallel shadow execution, named FPSanitizer, to find errors comprehensively on the multicore machines, while improving the efficiency of prior approaches. EFTSanitizer [38] further reduces the performance overheads by using error-free transformation.

Approximating error bounds based on static analysis methods. The methods mentioned above are all dynamic approaches, requiring actual code execution to identify errors. Another key thread of error detection is static analysis. Implemented using abstract semantic analysis [39], interval operations [40], affine operations [41], the static analysis tools [42], [43], [44], [45], [46] could provide error bounds, which usually exceed the actual errors.

Searching for error-inducing inputs. Recently, a series of works have been proposed to find inputs that lead to big errors. Chiang et al. [13] conducted binary guided random testing to search for maximum errors by using 64- or 128-bit floating-point arithmetic. Zou et al. [14] presented the first metaheuristic search-based method to automatically generate test inputs that trigger inaccuracies. They found that the fraction of exponents that cause significant relative errors is generally quite small, which is used for designing their search algorithm. Zou et al. [8] proposed ATOMU, a novel searching method based on mathematical conditions numbers. They believe floating-point errors could be attributed to large condition numbers of atomic operations, such as catastrophic cancellation. They use an evolutionary algorithm to find inputs which result in a huge condition number for each suspicious operation. Yi et al. [15] presented two algorithms, named DEMC and PTB, to detect bugs. DEMC utilizes the Differential Evolution algorithm and the Monte Carlo Markov Chain algorithm to detect inputs triggering maximum errors. PTB then generates the target input interval. FPGen was proposed by Guo et al. [16], which takes the topic as a code coverage maximization problem and solves the issue by using symbolic execution. Except for oracles calculated by using time-consuming high-precision programs, Wang et al. [17] introduced two other fitnesses, including approximate derivation function and condition number, to save time. Eiffel [18] first selects many samples uniformly and choose the cases

with large ULP errors. Then these data are assembled into clusters. Eiffel finally uses polynomial curves to report the maximal error. HSED [19] is a kind of hierarchical search algorithm to detect error-inducing inputs. The first layer is half precision, where HSED traverses all floating-point numbers exhaustively and record the substantial errors. As for the second layer, HSED traverses all the single precision tiny interval corresponding to each output of the first layer. With respect to the double precision layer, exhaustive search is not practical and random detection is conducted. FPCC [23], the most closely related tool, employs a combination of direct search and line search techniques. It effectively addresses the false positive issues inherent in ATOMU, demonstrating robust and satisfactory performance.

ATOMU, HSED, and FPCC are three state-of-the-art methods designed to detect error-inducing inputs. ICE can enhance their performance both effectively and efficiently. Since our method is designed to provide erroneous domains for other tools, we believe that it can be deployed across all tools, demonstrating potential to contribute to the entire field.

IX. CONCLUSION

Ensuring the correctness of floating-point programs is important. Automated approaches to identifying error-inducing inputs of floating-point programs have been proposed. Most of these approaches suffer from two limitations. First, based on search algorithms, existing approaches search from a large and sparse input space. Second, existing tools often rely on high-precision programs to act as an oracle, which is shown to provide unstable results. To provide reduced and highly qualified domains for these exciting approaches, we propose ICE based on simulated float-point errors. Experimental results demonstrate that ICE could significantly enhance the performance of the existing approach with reduced execution time.

In the future, we plan to further explore the potential of floating-point error simulation in ensuring the correctness of large and complex software programs. Since few studies have utilized complex programs as datasets, this future work holds significant value.

REFERENCES

- [1] X. He, X. Wang, J. Shi, and Y. Liu, "Testing high performance numerical simulation programs: Experience, lessons learned, and open issues," in *Proc. 29th ACM SIGSOFT Int. Symp. Softw. Testing Anal.*, 2020, pp. 502–515.
- [2] X.-P. Xu and A. Needleman, "Numerical simulations of fast crack growth in brittle solids," *J. Mech. Phys. Solids*, vol. 42, no. 9, pp. 1397–1434, 1994.
- [3] M. Kotrla, "Numerical simulations in the theory of crystal growth," *Comput. Phys. Commun.*, vol. 97, no. 1-2, pp. 82–100, 1996.
- [4] S. A. Orszag and M. Israeli, "Numerical simulation of viscous incompressible flows," *Annu. Rev. Fluid Mechanics*, vol. 6, no. 1, pp. 281–318, 1974.
- [5] T. Rives, M. Razack, J.-P. Petit, and K. Rawnsley, "Joint spacing: Analogue and numerical simulations," *J. Struct. Geol.*, vol. 14, no. 8-9, pp. 925–937, 1992.
- [6] R. Kobayashi, "Modeling and numerical simulations of dendritic crystal growth," *Physica D*, vol. 63, no. 3-4, pp. 410–423, 1993.
- [7] T. Bao and X. Zhang, "On-the-fly detection of instability problems in floating-point program execution," in *Proc. ACM SIGPLAN Int. Conf. Object Oriented Program. Syst. Lang. Appl.*, 2013, pp. 817–832.

- [8] D. Zou, M. Zeng, Y. Xiong, Z. Fu, L. Zhang, and Z. Su, "Detecting floating-point errors via atomic conditions," *Proc. ACM Program. Lang.*, vol. 4, no. POPL, pp. 1–27, 2019.
- [9] Y. Zhang, L. Ren, L. Chen, Y. Xiong, S.-C. Cheung, and T. Xie, "Detecting numerical bugs in neural network architectures," in *Proc. 28th ACM Joint Meeting Eur. Softw. Eng. Conf. Symp. Found. Softw. Eng.*, 2020, pp. 826–837.
- [10] M. Yan, J. Chen, X. Zhang, L. Tan, G. Wang, and Z. Wang, "Exposing numerical bugs in deep learning via gradient back-propagation," in *Proc. 29th ACM Joint Meeting Eur. Softw. Eng. Conf. Symp. Found. Softw. Eng.*, 2021, pp. 627–638.
- [11] J. Liu et al., "Automated detection and repair of floating-point precision problems in convolutional neural network operators," *ACM Trans. Softw. Eng. Methodol.*, vol. 34, no. 7, pp. 1–32, 2025.
- [12] J. Chen, C. Jia, Y. Yan, J. Ge, H. Zheng, and Y. Cheng, "A miss is as good as a mile: Metamorphic testing for deep learning operators," *Proc. ACM Softw. Eng.*, vol. 1, no. FSE, pp. 2005–2027, 2024.
- [13] W.-F. Chiang, G. Gopalakrishnan, Z. Rakamaric, and A. Solovyyev, "Efficient search for inputs causing high floating-point errors," in *Proc. 19th ACM SIGPLAN Symp. Princ. Pract. Parallel Program.*, 2014, pp. 43–52.
- [14] D. Zou, R. Wang, Y. Xiong, L. Zhang, Z. Su, and H. Mei, "A genetic algorithm for detecting significant floating-point inaccuracies," in *Proc. IEEE/ACM 37th IEEE Int. Conf. Softw. Eng.*, vol. 1, 2015, pp. 529–539.
- [15] X. Yi, L. Chen, X. Mao, and T. Ji, "Efficient automated repair of high floating-point errors in numerical libraries," *Proc. ACM Program. Lang.*, vol. 3, no. POPL, pp. 1–29, 2019.
- [16] H. Guo and C. Rubio-González, "Efficient generation of error-inducing floating-point inputs via symbolic execution," in *Proc. ACM/IEEE 42nd Int. Conf. Softw. Eng.*, 2020, pp. 1261–1272.
- [17] Z. Wang, X. Yi, H. Yu, and B. Yin, "Detecting high floating-point errors via ranking analysis," in *Proc. 29th Asia-Pacific Softw. Eng. Conf.*, 2022, pp. 397–406.
- [18] Z. Zhang et al., "Eiffel: Inferring input ranges of significant floating-point errors via polynomial extrapolation," in *Proc. 38th IEEE/ACM Int. Conf. Automated Softw. Eng.*, 2023, pp. 1441–1453.
- [19] Z. Zhang, J. Xu, J. Hao, Y. Qu, H. He, and B. Zhou, "Hierarchical search algorithm for error detection in floating-point arithmetic expressions," *J. Supercomputing*, vol. 80, no. 1, pp. 1183–1205, 2024.
- [20] L. Fousse, G. Hanrot, V. Lefèvre, P. Pélicier, and P. Zimmermann, "MPFR: A multiple-precision binary floating-point library with correct rounding," *ACM Trans. Math. Softw.*, vol. 33, no. 2, pp. 13–es, 2007.
- [21] T. Mpmath, development team, mpmath: A Python library for arbitrary-precision floating-point arithmetic (version 1.3.0), 2023. Accessed: Mar. 8, 2023. [Online]. Available: <http://mpmath.org/>
- [22] P. Larsson, "Exploring quadruple precision floating point numbers in GCC and ICC," 2013. [Online]. Available: <https://www.nsc.liu.se/~pla/blog/2013/10/17/quadruple-precision/>
- [23] X. Yi, H. Yu, L. Chen, X. Mao, and J. Wang, "FPCC: Detecting floating-point errors via chain conditions," *Proc. ACM Program. Lang.*, vol. 8, no. OOPSLA2, pp. 1504–1531, 2024.
- [24] M. L. Overton, *Numerical Computing with IEEE Floating Point Arithmetic*. Philadelphia, PA, USA: SIAM, 2001.
- [25] F. Jézéquel and J.-M. Chesneaux, "CADNA: A library for estimating round-off error propagation," *Comput. Phys. Commun.*, vol. 178, no. 12, pp. 933–955, 2008.
- [26] N. J. Higham, *Accuracy and Stability of Numerical Algorithms*. Philadelphia, PA, USA: SIAM, 2002.
- [27] D. S. Parker, P. R. Eggert, and B. Pierce, "Monte Carlo arithmetic: A framework for the statistical analysis of roundoff error," Computer Science Department, University of California (Los Angeles), 1997.
- [28] E. Tang, E. Barr, X. Li, and Z. Su, "Perturbing numerical calculations for statistical analysis of floating-point program (in) stability," in *Proc. 19th Int. Symp. Softw. Testing Anal.*, 2010, pp. 131–142.
- [29] C. Lattner and V. Adve, "LLVM: A compilation framework for lifelong program analysis & transformation," in *Proc. Int. Symp. Code Gener. Optim.*, 2004, pp. 75–86.
- [30] T. Bäck, *Evolutionary Computation 1: Basic Algorithms and Operators*. Boca Raton, FL, USA: CRC press, 2018.
- [31] N. Nethercote and J. Seward, "Valgrind: A framework for heavyweight dynamic binary instrumentation," *ACM Sigplan Notices*, vol. 42, no. 6, pp. 89–100, 2007.
- [32] F. Benz, A. Hildebrandt, and S. Hack, "A dynamic program analysis to find floating-point accuracy problems," *ACM SIGPLAN Notices*, vol. 47, no. 6, pp. 453–462, 2012.
- [33] F. François and L. Bruno, "VERROU: A CESTAC evaluation without recompilation," in *Proc. 17th Int. Symp. Sci. Comput., Comput. Arithmetic Verified Numerics (SCAN)*, 2016.
- [34] A. Sanchez-Stern, P. Panchekha, S. Lerner, and Z. Tatlock, "Finding root causes of floating point error," in *Proc. 39th ACM SIGPLAN Conf. Program. Lang. Des. Implementation*, 2018, pp. 256–269.
- [35] P. Panchekha, A. Sanchez-Stern, J. R. Wilcox, and Z. Tatlock, "Automatically improving accuracy for floating point expressions," *ACM Sigplan Notices*, vol. 50, no. 6, pp. 1–11, 2015.
- [36] W.-C. Lee, T. Bao, Y. Zheng, X. Zhang, K. Vora, and R. Gupta, "RAIVE: Runtime assessment of floating-point instability by vectorization," in *Proc. ACM SIGPLAN Int. Conf. Object-Oriented Program., Syst., Lang., IEEE Int. Symp. Spread Spectr. Tech. Appl.*, 2015, pp. 623–638.
- [37] S. Chowdhary and S. Nagarakatte, "Parallel shadow execution to accelerate the debugging of numerical errors," in *Proc. 29th ACM Joint Meeting Eur. Softw. Eng. Conf. Symp. Found. Softw. Eng.*, 2021, pp. 615–626.
- [38] S. Chowdhary and S. Nagarakatte, "Fast shadow execution for debugging numerical errors using error free transformations," *Proc. ACM Program. Languages*, vol. 6, no. OOPSLA2, pp. 1845–1872, 2022.
- [39] P. Cousot and R. Cousot, "Abstract interpretation: Past, present and future," in *Proc. Joint Meeting Twenty-Third EACSL Annu. Conf. Comput. Sci. Logic (CSL) Twenty-Ninth Annu. ACM/IEEE Symp. Logic Comput. Sci. (LICS)*, 2014, pp. 1–10.
- [40] T. Hickey, Q. Ju, and M. H. Van Emden, "Interval arithmetic: From principles to implementation," *J. ACM*, vol. 48, no. 5, pp. 1038–1068, 2001.
- [41] L. H. De Figueiredo and J. Stolfi, "Affine arithmetic: Concepts and applications," *Numer. Algorithms*, vol. 37, no. 1, pp. 147–158, Dec. 2004.
- [42] A. Izycheva and E. Darulova, "On sound relative error bounds for floating-point arithmetic," in *Proc. Formal Methods Comput. Aided Des. (FMCAD)*, 2017, pp. 15–22.
- [43] W. Lee, R. Sharma, and A. Aiken, "Verifying bit-manipulations of floating-point," in *Proc. 37th ACM SIGPLAN Conf. Program. Lang. Des. Implementation*, 2016, pp. 70–84.
- [44] A. Solovyyev, M. S. Baranowski, I. Briggs, C. Jacobsen, Z. Rakamarić, and G. Gopalakrishnan, "Rigorous estimation of floating-point round-off errors with symbolic taylor expansions," *ACM Trans. Program. Lang. Syst. (TOPLAS)*, vol. 41, no. 1, pp. 1–39, 2018.
- [45] E. Goubault and S. Putot, "Static analysis of finite precision computations," in *Proc. Int. Workshop Verification, Model Checking, Abstract Interpretation*, 2011, pp. 232–247.
- [46] S. Simić, A. Bemporad, O. Inverso, and M. Tribastone, "Tight error analysis in fixed-point arithmetic," *Formal Aspects Comput.*, vol. 34, no. 1, pp. 1–32, 2022.